



Project

Python Flask – Project

Building a Flask + MySQL Data Pipeline

Objective

In this assignment, you will continue working on your existing Flask project.

You will now add a **MySQL database** to your Flask application.

You will build two important data flows:

Data Flow 1 – Form → Flask → MySQL

User | | Fills out 10-question form ↓HTML Form | | POST ↓Flask | | Read form data | Proc

Data Flow 2 – MySQL → Flask → HTML

MySQL Database | | SELECT ↓Flask | | Process database data ↓HTML Template | ↓Browser | ↓

Part 1: Continue Your Existing Project

Do **not** create a new Flask project.

Continue working on the Flask project from the previous assignment.

Your existing project should currently look similar to:

```
flask_in_class_assignment/| |— venv/| |— main.py| |— templates/ | |— home.html | |—
```

You will now add MySQL-related functionality.

Part 2: Install MySQL Connector

Make sure your virtual environment is activated.

Install the MySQL connector:

```
pip install mysql-connector-python
```

Verify that it was installed:

```
pip list
```

You should see:

```
mysql-connector-python
```

You will use this package to allow Flask/Python to communicate with MySQL.

Part 3: Create a MySQL Database

Open MySQL Workbench or another MySQL client.

Create a database called:

```
CREATE DATABASE flask_assignment;
```

Select the database:

```
USE flask_assignment;
```

Part 4: Create the Database Table

Your form will collect information about a user.

The form must contain **at least 10 questions**.

Each question should have a corresponding column in the MySQL table.

Form Questions

Use the following 10 questions:

#	Question	Form Field	MySQL Column	Da
1	What is your full name?	full_name	full_name	VARC
2	What is your email address?	email	email	VARC
3	How old are you?	age	age	INT
4	What city do you live in?	city	city	VARC
5	What is your country?	country	country	VARC
6	What is your occupation?	occupation	occupation	VARC
7	What is your favourite hobby?	hobby	hobby	VARC
8	What is your favourite programming language?	programming_language	programming_language	VARC
9	How many hours per week do you spend online?	online_hours	online_hours	INT
10	What is your favourite type of entertainment?	entertainment	entertainment	VARC

The database will also have an automatically generated ID.

Part 5: Create the MySQL Table

Run the following query:

```
CREATE TABLE users (    id INT AUTO_INCREMENT PRIMARY KEY,    full_name VARCHAR(100),
```

Your table should contain:

```
id full_name email age city country occupation hobby programming_language online_hours €
```

Part 6: Understand the Connection

Your Flask application now needs to connect to MySQL.

The basic connection will look like:

```
import mysql.connector
db = mysql.connector.connect(
    host="localhost",
    user="root",
    password="YOUR_PASSWORD",
    database="flask_assignment")
```

Replace:

YOUR_PASSWORD

with your MySQL password.

Important

Do not share your MySQL password with other students.

Part 7: Update the Form

Go back to your existing:

templates/form.html

The form should now contain the 10 questions.

For example:

```
<form action="/submit-form" method="POST">
  <label>Full Name:</label>    <input type="text" name="full_name">
  <label>Email:</label>    <input type="email" name="email">
  <label>Age:</label>    <input type="number" name="age">
  <label>City:</label>    <input type="text" name="city">
  <label>Country:</label>    <input type="text" name="country">
  <label>Occupation:</label>    <input type="text" name="occupation">
  <label>Hobby:</label>    <input type="text" name="hobby">
  <label>Favourite Programming Language:</label>    <input type="text" name="programm
  <label>Hours spent online per week:</label>    <input type="number" name="online_ho
  <label>Favourite Entertainment:</label>    <input type="text" name="entertainment">
  <button type="submit">Submit Form</button>
</form>
```

Important Concept

Notice that the name attribute matches the database column.

For example:

```
<input type="text" name="full_name">
```

matches:

```
full_name
```

in MySQL.

This makes it easier for Flask to process the data.

Part 8: Receive the Form Data in Flask

Your existing route:

```
@app.route("/submit-form", methods=["POST"])def submit_form():
```

will now receive the form data.

Use:

```
full_name = request.form.get("full_name")email = request.form.get("email")age = request.
```

At this stage, Flask has received the information from the browser.

Part 9: Process the Data in Flask

Before sending the data to MySQL, Flask should process the data.

For example, numeric form values arrive as strings.

So:

```
age = request.form.get("age")
```

gives you something like:

```
"25"
```

You can convert it to an integer:

```
age = int(request.form.get("age"))
```

Similarly:

```
online_hours = int(request.form.get("online_hours"))
```

You can also remove unnecessary spaces:

```
full_name = request.form.get("full_name").strip()city = request.form.get("city").strip()
```

This demonstrates the idea of:

Receive → Process → Store

Part 10: Insert the Data into MySQL

After processing the form data, create an SQL INSERT query.

```
query = """    INSERT INTO users    (full_name, email, age, city, country, occupation,
```

Then execute the query:

```
cursor = db.cursor()
cursor.execute(query, (    full_name,    email,    age,    city,    country,    occupati
db.commit()
```

The important part is:

```
db.commit()
```

This saves the inserted record to the database.

Part 11: Verify the Data in MySQL

After submitting the form, go to MySQL and run:

```
SELECT * FROM users;
```

You should see the submitted information.

For example:

```
1 | John Smith | john@email.com | 25 | Toronto | Canada | Developer | Gaming | Python |
```

Each form submission should create a **new row**.

Part 12: Create a New Endpoint to Fetch Data

Now you will build the second part of the data pipeline.

Create a new Flask endpoint:

```
/users
```

The URL will be:

```
http://127.0.0.1:5000/users
```

This endpoint should:

1. Connect to MySQL
2. Run a SELECT query
3. Fetch the data
4. Process the data in Flask
5. Send the data to an HTML template
6. Display the data in the browser

The flow is:

Browser | GET /users ↓ Flask | SELECT * FROM users ↓ MySQL | User records

Part 13: Write the SELECT Query

The SQL query will be:

```
SELECT * FROM users;
```

In Flask:

```
cursor = db.cursor()
query = "SELECT * FROM users"
cursor.execute(query)
users = cursor.fetchall()
```

`fetchall()` returns all the records.

For example:

```
[ (1, "John Smith", "john@email.com", 25, "Toronto", ...), (2, "Sarah Lee", "sarah@email.com", 30, "New York", ...) ]
```

Part 14: Create users.html

Create a new HTML file:

templates/ | home.html | contact.html | form.html | users.html

The page should display the users in a table.

Example:

```
<h1>Registered Users</h1>
<table border="1">
  <tr>
    <th>ID</th>
    <th>Name</th>
    <th>Email</th>
    <th>Age</th>
  </tr>
  {% for user in users %}
  <tr>
    <td>{{ user[0] }}</td>
    <td>{{ user[1] }}</td>
    <td>{{ user[2] }}</td>
    <td>{{ user[3] }}</td>
  </tr>
  {% endfor %}
</table>
```

This introduces students to using Flask/Jinja data inside an HTML template.

Part 15: Send Data From Flask to HTML

The `/users` endpoint should eventually do something like:

```
@app.route("/users")def users():
    cursor = db.cursor()
    query = "SELECT * FROM users"
    cursor.execute(query)
    users = cursor.fetchall()
    return render_template("users.html", users=users)
```

The important part is:

```
render_template("users.html", users=users)
```

Here:

```
users
```

is the Python variable containing the database records.

It is being passed to:

```
users.html
```

Part 16: Test the Complete Pipeline

Students should test both pipelines.

Test 1 – Submit Data

Open:

```
http://127.0.0.1:5000/form
```

Fill out all 10 questions.

Click:

```
Submit Form
```

The data should be inserted into MySQL.

Check:

```
SELECT * FROM users;
```

Test 2 – Display Data

Open:

`http://127.0.0.1:5000/users`

The users stored in MySQL should appear on the HTML page.

Final Project Structure

When finished, the project should look approximately like:

```
flask_in_class_assignment/|— venv/|— main.py|— templates/    |— home.html    |—
```

Final Routes

The application should now have:

Page	Method	URL
Home	GET	/
Contact	GET	/contact
Form	GET	/form
Submit Form	POST	/submit-form
Display Users	GET	/users

0 % 0 of 1 topics complete

Project

 Assignment